mechanical-work tools — user guide

Revision: 1 **Last modified:** 2026-09-08T20:24:07Z **Authority:** constitution §11.4.274 (mechanical work belongs in a script) · §11.4.273 (measuring-instrument verification) · §11.4.201 (a guard asserts the REAL condition) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for every script under `constitution/scripts/mechanical/`

What this document is, and what it is not

This is the **task-oriented user guide**: how to reach for the right tool, what a correct invocation looks like, and how to read what comes back. The **reference manual** — every flag, every failure path, the forensic notes behind each design choice — lives beside the code at `constitution/scripts/mechanical/README.md` and is not restated here (§11.4.227: extend, do not duplicate).

Read this first when you are about to do one of the loops below by hand. Read the README when you need the exact semantics of a flag.

The six tools, and the question each answers

Tool	The question	Reach for it when
`mutation_harness.sh`	"Does this gate actually fail when its subject breaks?"	You are pairing a §1.1 mutation to a gate, or auditing an existing suite
`census_query.sh`	"How many, and can this query even see?"	Any count, grep or inventory whose answer will drive a decision
`suite_fanout.sh`	"What moved since the last round?"	Running N suites and comparing against a baseline
`residue_scan.sh`	"Did a mutation marker leak into the tree?"	Before any commit that followed a mutation round (§11.4.84)
`anchor_census.sh`	"Do all carriers declare the same anchor set?"	Governance cascade / lockstep checks (§11.4.157 / §11.4.227(B))
`await_condition.sh`	"Has it finished yet — and did it finish, or did I give up?"	Polling for a bounded completion

They are **inherited by reference** (§11.4.177): invoke them from wherever the constitution is checked out. Never copy one into a project, and never hardcode a project path into one — every tool takes its target as an argument.

The exit-code vocabulary — read this before reading any result

Every tool uses the same three-value vocabulary, and the whole point is that they are **three different facts**:

Code	Meaning	The mistake it prevents
`0`	ran; outcome matched expectation	—
`1`	ran; **found something** (a finding)	reading a real finding as an error and dismissing it
`2`	**could not run** — bad args, missing target, a failed control	reading "could not look" as "found nothing"
`3`	a bounded wait **expired** (`await_condition.sh` only)	reading a timeout as success

A tool that collapsed `0` and `2` would turn a broken instrument into a clean bill of health. That is the failure this vocabulary exists to make impossible, and it is why none of these tools has a `--quiet-failures` style flag.

1 1

Task: prove a gate is real (paired § . mutation)

The loop — copy the tree, apply a one-line edit, run the suite, read the summary, restore, verify the restore — is deterministic and decision-free, so it belongs in a script rather than in an agent's context (§11.4.274).

```
MECH=/path/to/constitution/scripts/mechanical

# 1. BASELINE first. A mutation result means nothing without it.
"$MECH/mutation_harness.sh" --tree /path/to/repo \
  --suite scripts/tests/test_thing.sh --name BASELINE --expect pass

# 2. Then the mutation. --expect fail says what you believe; a VIOLATION means
#    the gate survived its own mutation and is decoration.
"$MECH/mutation_harness.sh" --tree /path/to/repo \
  --suite scripts/tests/test_thing.sh --name M1 --expect fail \
  --file src/subject.sh --from 'exact literal text' --to 'replacement'
```

Read the verdict line:

```
MUTATION M1: EXPECTED-FAIL got 23 failed / 35 passed [OK]
  suite=... mode=scratch-copy replacements=1 suite_rc=1
  parsed_line=✗ 23 failed, 35 passed
  tree_fingerprint_unchanged=a3618d114d51a2b0962f6dc3f853b847
```

- `[OK]` — the outcome matched `--expect`.
- `[VIOLATION]` — it did not. **This is the finding.** A mutation that leaves a suite green means the suite does not guard the thing you mutated.
- `replacements=1` — the mutation was actually applied. `0` never gets here: an absent `--from` exits `2` rather than reporting a suite result, because a result from an unapplied mutation describes nothing.
- `tree_fingerprint_unchanged=` — proof your real tree was not modified.

Two traps worth knowing before you trust a number. First, harness summary fields swap order between pass and fail (`58 passed, 0 failed` vs `23 failed, 35 passed`), so positional parsing reads a failing run as 23 passes — each count is located by its own label instead. Second, `--env-knob` runs the suite from the **original** tree; a suite that writes an artifact beside itself will write into your real repository. Prefer the default scratch-copy mode.

Task: count something without lying about it

Every count that will drive a decision carries a needle it MUST find and a needle it MUST NOT (§11.4.273). Both are mandatory; the tool refuses to run without them, because optional controls decay into a convention and a convention is what had already failed when the anchor was minted.

```
"$MECH/census_query.sh" --tree ./docs --pattern 'mutation_harness\.sh' \
  --positive 'AGENT_GUARDRAILS' \
  --negative 'zzz_fabricated_needle' \
  --label harness_in_docs
```

```
CENSUS harness_in_docs: matches=0 files=0 positive=1/1 negative=0/1 [OK]
```

That zero is **trustworthy**, and the reason is on the same line: `positive=1/1` says a value known present was found through this exact query path, and `negative=0/1` says a fabricated value was not. Without those two counts, a zero and a broken grep are the same output.

Choosing the needles.

- The positive needle must lie **inside the query's own `--include` scope**. A needle that exists in the tree but outside the scope produces a false "instrument is blind" signal — so the tool detects that case and says `POSITIVE CONTROL OUT OF SCOPE` instead, which is a different problem with a different fix (move the control, not the query).
- The positive needle must **not be the thing you are measuring**. A control that is the question under test cannot discriminate.
- When the criterion is a **set**, pass every member (§11.4.273(f)). `--positive` and `--negative` are both repeatable and every one must hold. A two-literal denylist searched with one literal passes both controls while the answer is wrong — measured, and it reported a live credential eliminated while it was still there.

Add `--expect-count N` when you know what the answer should be: a mismatch exits `1` (a finding) while still printing the measured count, which is distinct from the instrument failing to measure at all (`2`).

Task: wait for something to finish

```
"$MECH/await_condition.sh" --until 'test -f /tmp/build.done' \
  --timeout 600 --interval 5 --label build
```

Exactly one outcome line, and the two outcomes are not interchangeable:

```
AWAIT build: SATISFIED after 41s (polls=9)          # exit 0
AWAIT build: TIMEOUT after 600s (polls=121) — condition never held   #
exit 3
```

A timeout exits `3`, never `0`, so no caller can accidentally treat "I gave up" as "it finished". `--pid-gone PID` uses `kill -0`, which tests existence without signalling — this tool never signals a process (§11.4.174).

Task: check nothing leaked before a commit

```
"$MECH/residue_scan.sh" --tree /path/to/repo --control-needle 'a string really in the tree'
```

Exit `1` means residue was found (a finding, act on it). Exit `2` means the scan could not see — including the case where every file was excluded, which the tool refuses to report as `residue=0`. Legitimate self-references (a rule that *defines* a marker, a mutation test that *plants* one) go in an allowlist, and the allowlist is itself audited: patterns matching nothing are reported `STALE-ALLOW`, so it cannot quietly grow until it disables the scan.

Task: verify governance carriers are in lockstep

```
"$MECH/anchor_census.sh" --opener-re '^### §11\.4\.[0-9]+' \
  --control-needle 11.4.274 Constitution.md
```

It counts anchors that **open a block**, not anchors merely cited — a carrier references far more than it declares, and conflating the two produced a report of 900 "anchors" and a wall of phantom duplicates from 271 real openers. Narrow `--opener-re` per carrier class; the README's table says which shape to use where.

Running the tools' own tests

constitution/scripts/mechanical/tests/run_all.sh

Hermetic — each suite builds its own fixture tree and its own suites, with no dependency on any consuming project's harness. Measured 2026-09-08: `ALL SUITES: 181 passed, 0 failed`, and `212 passed, 0 failed` after the `census_query.sh` suite landed.

A consuming project verifies the tools from its own side. The tools' suites are deliberately project-agnostic, so an acceptance test that reproduces a *measured outcome of a specific repository* belongs to that repository, reaching out to the constitution rather than the other way round. The reference example is `claude_toolkit/scripts/tests/test_mechanical_tools_acceptance.sh`, which checks each tool's refusal paths and — opt-in via `CMA_MECH_ACCEPTANCE=1` — reproduces that repo's own baseline and a named mutation against it.

Honest boundaries (§ . .)

- These tools **gather and execute**. None decides whether a mutation is a good mutation, whether a gate is genuine, whether a fix is correct, or what a count means. Reading a result as evidence is the agent's job; a script that rendered that verdict would be the bluff gate this library exists to prevent (§11.4.274(d)).
- Control needles prove an instrument can tell PRESENT from ABSENT. They do **not** prove it measures the RIGHT property. A correctly-needled census of the wrong thing is still the wrong answer.
- `mutation_harness.sh` proves a suite's counts changed under a named edit. That the suite therefore guards the *right* invariant is a judgement, not an output.

Related

- Reference manual: `constitution/scripts/mechanical/README.md`
- `constitution/Constitution.md` §11.4.274, §11.4.273, §11.4.201, §11.4.84, §11.4.174